ארכיטקטורת חישוב בהשראת המוח – Neural Engineering Framework

מגיש: שלומי בן שושן

שאלה 1 – ייצוג נתונים באמצעות NEF

סעיף א':

תארו בקצרה בהתייחס ל-NEF, מהם Node, Ensemble ו-Connection.

תשובה:

Neural Engineering Framework, או NEF בר"ת, היא מסגרת חישובית שמשמשת למידול מערכות נוירוניות. NEF מספקת שלושה עקרונות יסוד המנחים בניית מודלים נוירוניים בקנה מידה גדול. בניגוד למידול המסורתי מלמטה-למעלה, NEF נוקטת בגישה מלמעלה-למטה, בה תיאורים ברמה הגבוהה של הרשת מאלצים תיאורים ברמה הנמוכה שלה. באמצעות NEF ניתן להדגים מערכות דינמיות מורכבות מאוד.

NEF מציעה שלושה עקרונות המנחים עיצוב מודלים בקנה מידה גדול:

1. **ייצוג** – שימוש באוכלוסייה (או אנסמבל) של נוירונים כדי לייצג אותות באופן מבוזר על-ידי קידוד ופענוח של נתונים לא ליניאריים, מבוססי ספייקים. בשלב הקידוד, מבנים מתמטיים מיוצגים על-ידי רכבות ספייקים. בשלב הפענוח, רכבות הספייקים מסוננות ונסכמות באופן ממושקל. ניתן לבצע אופטימיזציה של משקולות כדי לבנות מחדש את האות המיוצג או כדי להשיג טרנספורמציה רצויה.
2. **טרנספורמציה/חישוב** – מימוש פונקציות ליניאריות ולא-ליניאריות של אותות מקודדים באמצעות קשרים סינפטיים ממושקלים בין אנסמבלים של נוירונים.
3. **דינמיקה** – חיבור אנסמבלים לעצמם בסינפסות ממושקלות (חיבורי feedback) לטובת יצירה של מערכת דינמית שמקיימת פעילות נוירונית ומספקת פונקציונליות ב-high-level, כמו זיכרון.

השימוש ב-NEF יכול להיעשות דרך ספריית Python בשם Nengo שמספקת API למסגרת. באמצעות Nengo ניתן להגדיר אנסמבלים של נוירונים וקשרים בין האנסמבלים הללו. היא מספקת פונקציות קידוד, פענוח וטרנספורמציה שממומשות ב-backend של המסגרת, מה שמאפשר למפתח ולחוקר להתמקד במודל הנוירוני.

הרכיבים העיקריים ב-Nengo כוללים צמתים, אנסמבלים וקשרים, שבאים לידי ביטוי באמצעות המחלקות Node, Ensemble ו-Connection, בהתאמה. להלן הסבר קצר על כל אחת מהמחלקות:

1. **Node** – מחלקה ב-Nengo שמספקת קלט לא נוירוני למודל הנוירוני ומאפשרת לעבד את הפלטים שלו. באמצעות הצומת ניתן להזין את המודל בנתונים חיצוניים, ובכך לספק גירויים למודל, או לבצע חישובים שרירותיים לצורך שליטה בסימולציית Nengo. יש לציין שצמתים לרוב לא מהווים חלק מהמודל הנוירוני עצמו, אלא מייצגים משתנים סביבתיים שמקיימים אינטראקציה עם המודל.
2. **Ensemble** – מחלקה ב-Nengo שמממשת אוכלוסייה (קבוצה) של נוירונים שפועלים יחד כדי לקודד ולפענח מידע (לייצג ווקטור). לכל נוירון באנסמבל מוגדרת עקומת כוונון (Tuning Curve) שקובעת את

פעילות הנוירון (תדירות ירי הספייקים שלו) כתגובה לגירויים המתקבלים כקלט. האנסמבלים הם יחידות החישוב המרכזיות ב-NEF, והם משמשים לייצוג משתנים, ביצוע חישובים נוירוניים, ייצוג שגיאות, המרת אותות באמצעות קידוד ופענוח, ועוד.

3. **Connection** – מחלקה ב-Nengo שמחברת בין שני אובייקטים (למשל אנסמבלים או צמתים), ומגדירה את אופן העברת האותות ביניהם. החיבור מפרט כיצד פעילות בחלק אחד של המודל, שנקרא pre, משפיעה על פעילות בחלק אחר שלו, שנקרא $post$. החיבורים מממשים אינטראקציות סינפטיות בין נוירונים, מציינים את המשקולות הסינפטיות, וקובעים כיצד האותות עוברים טרנספורמציה.

יחד, רכיבים אלה מאפשרים למפתח ולחוקר לבנות ולדמות מערכות נוירוניות מורכבות מאוד על-ידי הגדרת האופן שבו מידע מיוצג, מעובד וזורם ברשת נוירונים.

<u>סעיף ב'</u>:

הסבירו מהן פונקציות הבסיס ומה חשיבותן עבור ייצור באמצעות NEF.

<u>תשובה</u>:

ב-NEF, פונקציות בסיס משמשות לייצוג וטרנספורמציה של מידע ברשתות נוירונים. הן משמשות לקידוד ופענוח של מידע באנסמבלים של נוירונים, ולטרנספורמציה של פעילויות נוירוניות.

בדומה לאופן בו ווקטורי בסיס מגדירים מרחב ווקטורי, פונקציות בסיס מגדירות מרחב פונקציונלי. למשל, באמצעות ניתוח פורייה, פונקציות ניתנות לפירוק או לקירוב על-ידי סכומים של פונקציות טריגונומטריות כמו סינוסים וקוסינוסים. לכל מרחב פונקציונלי ניתן למצוא פונקציות בסיס שפורשות אותו.

ב-NEF, עקומות הכוונון של הנוירונים משמשות כפונקציות בסיס מלא עבור מרחב הפונקציות שניתן לחשב באמצעות אותם נוירונים, ולכן הן חשובות מאוד עבור NEF. מכיוון שתגובות הנוירונים באנסמבל מייצגות בסיס מלא, הן לא בלתי-תלויות, ולכן רק חלק קטן מתת-המרחב של ערכי הפעילות הנוירונית מושגים באמצעות אותו אנסמבל. אם כן, באמצעות פונקציות בסיס ניתן לקודד מידע, והדרך לעשות זאת היא באמצעות עקומות הכוונון. אנסמבל של נוירונים מייצג את הקלט כאינטגרציה ממושקלת של עקומות הכוונון של הנוירונים בו. כמו כן, לנוירונים מסוימים, במיוחד במרחבים בממדים גבוהים, יש ווקטורי כיוון מועדפים שנותנים עדיפות בתגובה לגירויים מסוימים על-פני אחרים. על-ידי אינטגרציה של פעילויות הנוירונים עם כיוונים מועדפים שונים, האנסמבל יכול לייצג ביעילות אותות קלט מורכבים.

בנוסף, באמצעות פונקציות בסיס ניתן לפענוח מידע. הדרך לעשות זאת היא באמצעות פונקציות שחזור, המפענחות או משחזרות את האות המיוצג על-ידי פעילויות נוירוניות. תהליך הפענוח מאפשר לחלץ את המידע המקודד על-ידי אנסמבל ולנצל אותו לצורך חישובים נוירוניים או פעולות אחרות.

אם כן, חשיבותן של פונקציות הבסיס ב-NEF באה לידי ביטוי במספר מישורים:

1. <u>ייצוג מידע</u> – פונקציות בסיס מאפשרות לקודד משתנים רציפים תוך שימוש בפעילויות נוירוניות בדידות, מה שמאפשר לייצג מידע מורכב בעל ממדים גבוהים באופן מבוזר על-פני אנסמבל של נוירונים.

2. חישוב נוירוני – החישוב הנוירוני מסתמך על טרנספורמציות לינאריות ולא-לינאריות של אותות מקודדים. פונקציות הבסיס תומכות בחישובים כאלה על-ידי מיפוי קלטים לפעילות נוירונים ולהפך.

3. מינימיזציה של שגיאה – ניתן למזער שגיאות ייצוג וחישוב באמצעות אופטימיזציה של בחירת פונקציות הבסיס והמשקולות התואמות.

לסיכום, ב-Neural Engineering Framework, פונקציות בסיס משמשות לקידוד, טרנספורמציה ופענוח של מידע באוכלוסיית נוירונים. הן מספקות דרך לייצוג משתנים רציפים תוך שימוש בפעילויות נוירוניות בדידות, והן נדרשות לביצוע חישובים נוירוניים. באמצעות אופטימיזציה של פונקציות אלה, ניתן לייצג מידע ולבצע חישובים נוירוניים שמאפשרים סימולציה סבירה ביולוגית של דינמיקה נוירונית לטובת הבנת תפקוד המוח.

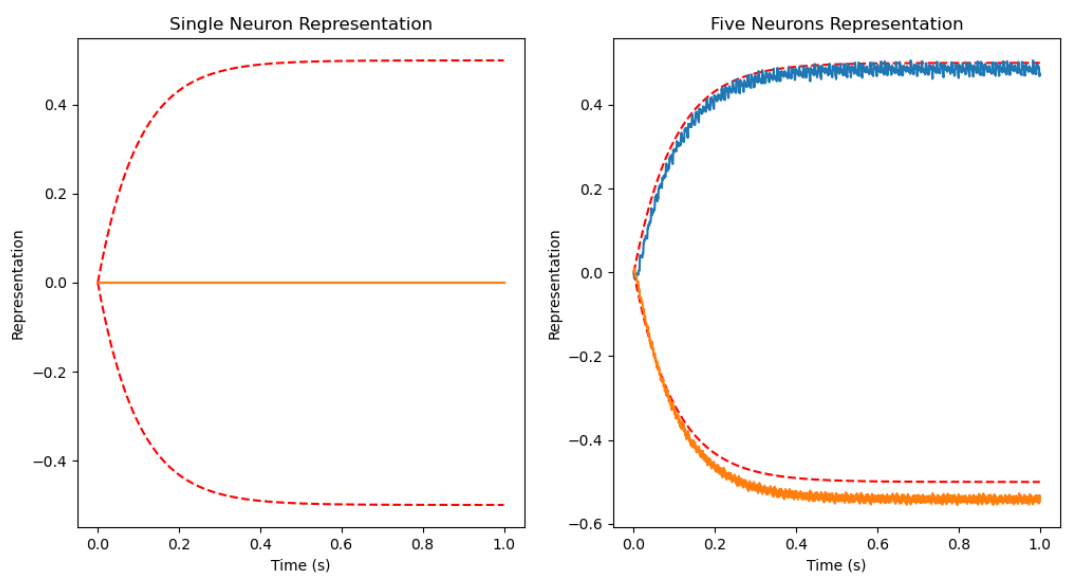
סעיף ג':

הדגימו באמצעות plots ו-Nengo שלושה מקרים בהם לא מספיק להשתמש בנוירון אחד לצורך ייצוג, והסבירו כל אחד מהמקרים.

תשובה:

1. ייצוג של גירוי בממד גבוה:

ננסה לשחזר גירוי דו-ממדי $\boldsymbol{x} = (0.5, 0.5)$ באמצעות נוירון בודד ובאמצעות חמישה נוירונים ונקבל:



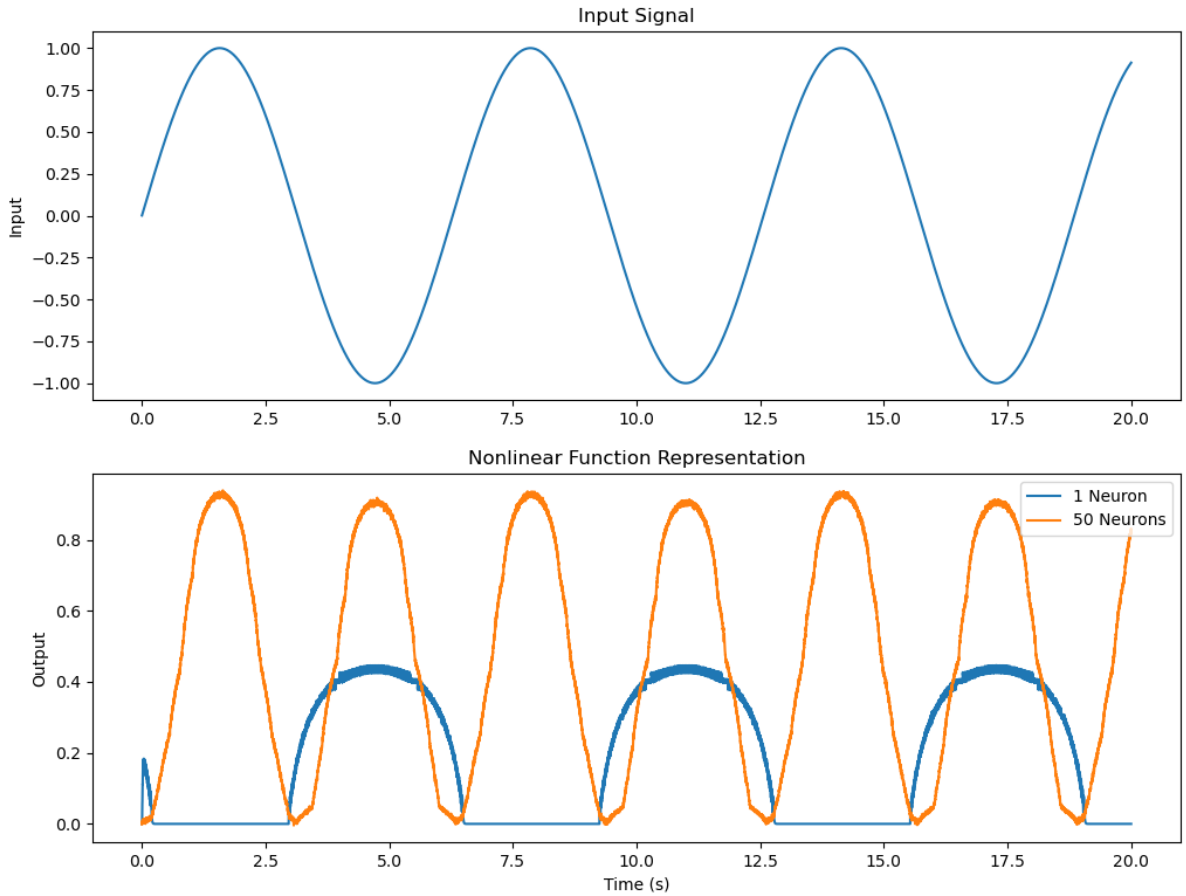
הקווקו האדום מתאר את גירוי הקלט. בסימולציה זו הוגדר $f(x) = x$, שכן ברצוננו לשחזר את אות הקלט. ניתן לראות שמודל בעל נוירון אחד לא מצליח לשחזר את הקלט כלל, בעוד שמודל בעל חמישה נוירונים מצליח לשחזר אות מקורב לקלט (האמנם לא זהה לאות הקלט, אך הוספה של נוירונים עשויה לשפר את הדיוק).

להלן קישור לקוד הסימולציה:

github.com/shlomi1993/Neuromorphic-Engineering/blob/main/nengo/HW/high_dim_stim.py

2. <u>ייצוג אות לא-לינארי</u>:

ננסה לקודד את הפונקציה $f(x) = x^2$ עבור קלט סינוסי בתחום $[-1,1]$, באמצעות נוירון יחיד ובאמצעות 50 נוירונים, ונקבל:



בחלק העליון של התרשים מוצג אות גירוי הקלט הסינוסי, ובחלק התחתון מוצג הפיענוח של האות בידי המודלים לאחר טרנספורמציה של x^2. ניתן לראות שהמודל בעל הנוירון היחיד לא מייצג את הפונקציה בהצלחה, בעוד שהמודל בעל 50 הנוירונים מציג תוצאה די טובה של פלט הפונקציה.

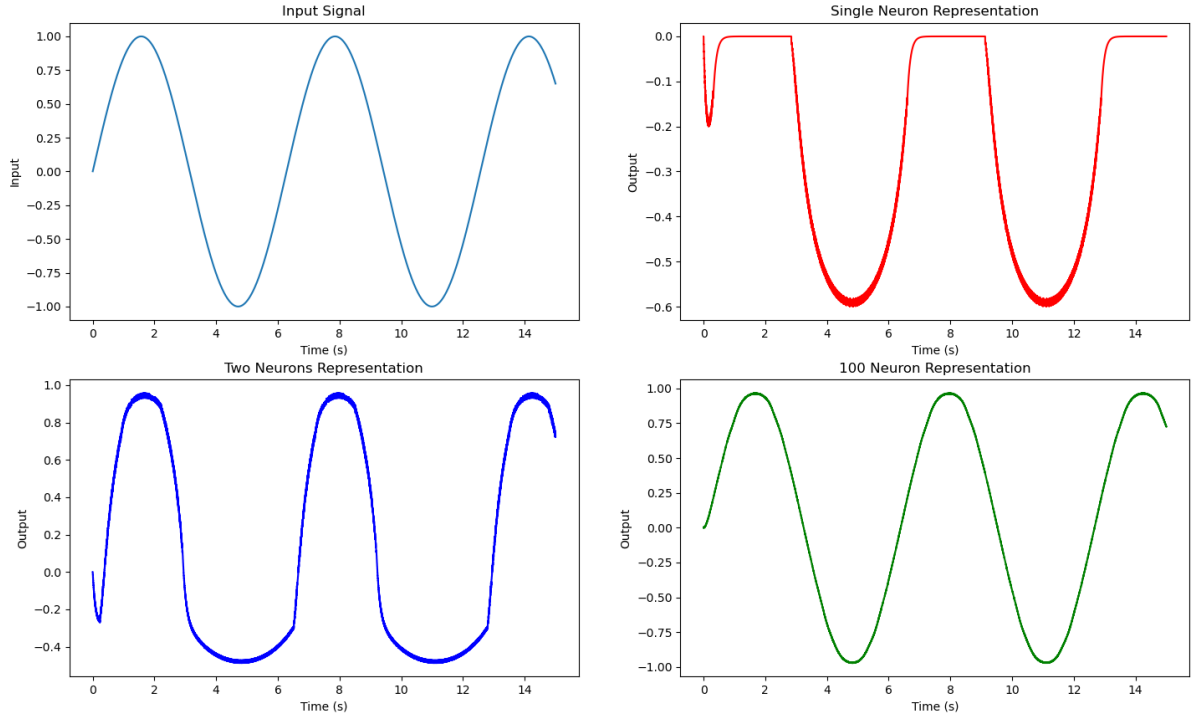
להלן קישור לקוד הסימולציה:

github.com/shlomi1993/Neuromorphic-Engineering/blob/main/nengo/HW/non_linear.py

[המשך בעמודים הבאים...]

3. <u>מקרה 3 – ייצוג מדויק של אות</u>:

מודל בעל נוירון יחיד עלול להיות מוגבל בייצוג מדויק של אות. ננסה לייצג אות סינוסי באמצעות נוירון יחיד, באמצעות שני נוירונים ובאמצעות 50 נוירונים, ונקבל:



בצד שמאל למעלה ניתן לראות את הקלט הסינוסי שמקבל במודל. בצד ימין למעלה ניתן לראות את שחזור האות על-ידי נוירון אחד. נראה שהנוירון הבודד לא מצליח לשחזר את אות הקלט כלל. בצד שמאל למטה ניתן לראות ייצוג על-ידי שני נוירונים. שחזור זה דומה מאוד לפונקציית סינוס, אך אינו מדויק. בצד ימין למטה ניתן לראות את השחזור על-ידי 100 נוירונים, וניכר שהוא יחסית נאמן למקור.

אם כן, נוירון יחיד לא יכול לייצג באופן מדויק אות גירוי קלט שהוא יותר מורכב מקבוע. נדרשים לפחות שני נוירונים לשם כך, ולפי הסימולציה לעיל נראה שהדיוק עולה ככל שמוסיפים נוירונים.

להלן קישור לקוד הסימולציה:

github.com/shlomi1993/Neuromorphic-Engineering/blob/main/nengo/HW/accuracy.py

[המשך בעמודים הבאים...]

סעיף ד':

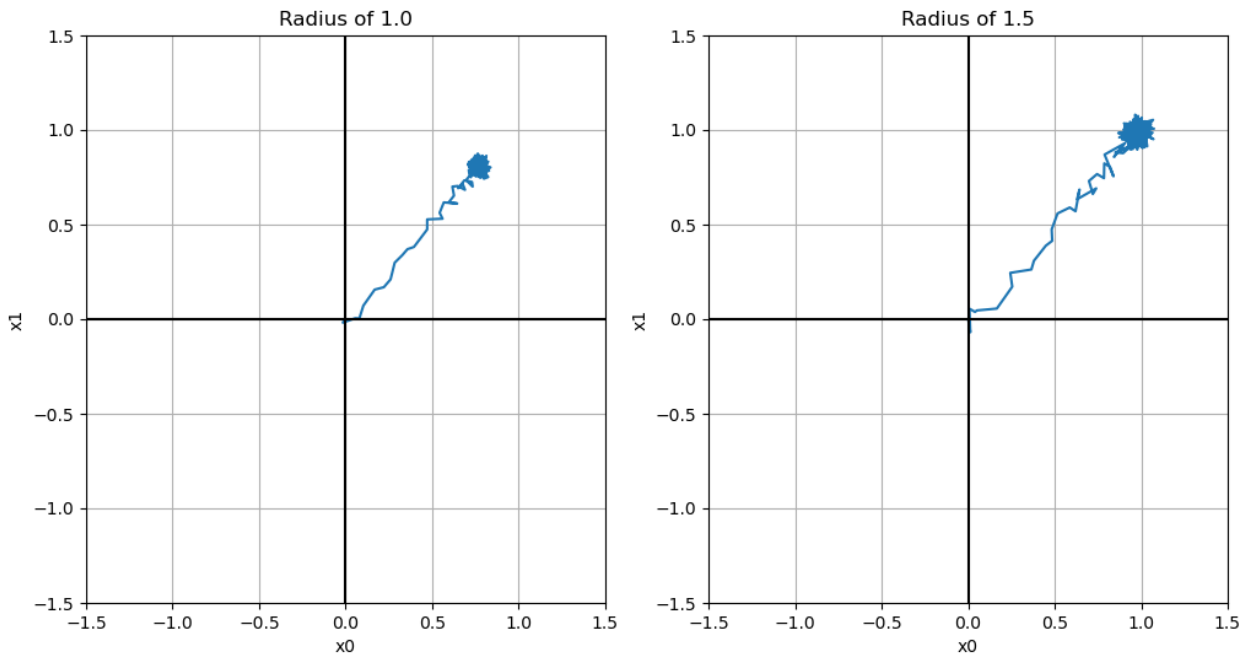
הסבירו מהו הרדיוס ומדוע הוא חשוב, והדגימו את חשיבותו על-ידי שימוש ב-Nengo ויציאה של plot רלוונטי בליווי הסבר.

תשובה:

ב-NEF, הרדיוס הוא פרמטר שמגדיר את קנה המידה של הקלט למרחב הקידוד של אנסמבל נוירונים. הוא זה שקובע את טווח הערכים שהאוכלוסייה כולה יכולה לייצג. הגדרה נכונה של הרדיוס חיונית להשגת ייצוגים נוירוניים מדויקים וסבירים ביולוגית. אם ננסה לקודד אות (נקודה n-ממדית כלשהי) שנמצאת מעבר לרדיוס, לא נקבל ייצוג מדויק שלו, ללא תלות במספר הנוירונים בהם נשתמש.

כך למשל, בהינתן קלט דו-ממדי $(1,1)$, אם הרדיוס הוא 1, אזי שלא ניתן לייצג את הקלט בדיוק גם באמצעות מספר גבוה של נוירונים, כי הנקודה $(1,1)$ נמצאת מעבר לרדיוס (מחוץ למעגל היחידה). אם נגדיל את הרדיוס ל-1.5, נקבל ייצוג תקין של הקלט.

להלן תרשים שמתאר את האמור לעיל:



התרשים נוצר על-ידי יצירה של שני אנסמבלים a ו-b, כל אחד כולל 100 נוירונים, כאשר הרדיוס של a הוא 1, והרדיוס של b הוא 1.5. אל שני האנסמבלים חוברה צומת $stim$ עם גירוי נקודתי $(1,1)$.

להלן קישור לקוד הסימולציה:

github.com/shlomi1993/Neuromorphic-Engineering/blob/main/nengo/HW/radius.py

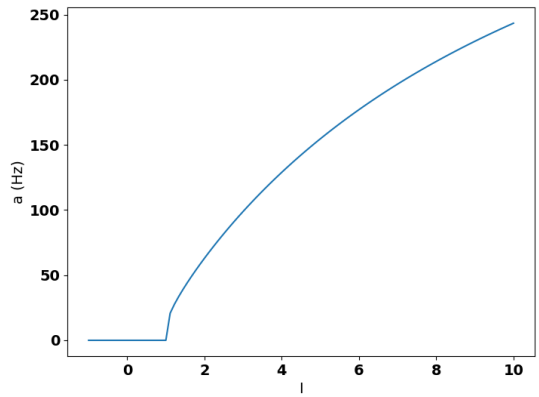
סעיף ה':

מהן Tuning Curves ב-Nengo? הסבירו מהם באמצעות plots ו-Nengo.

תשובה:

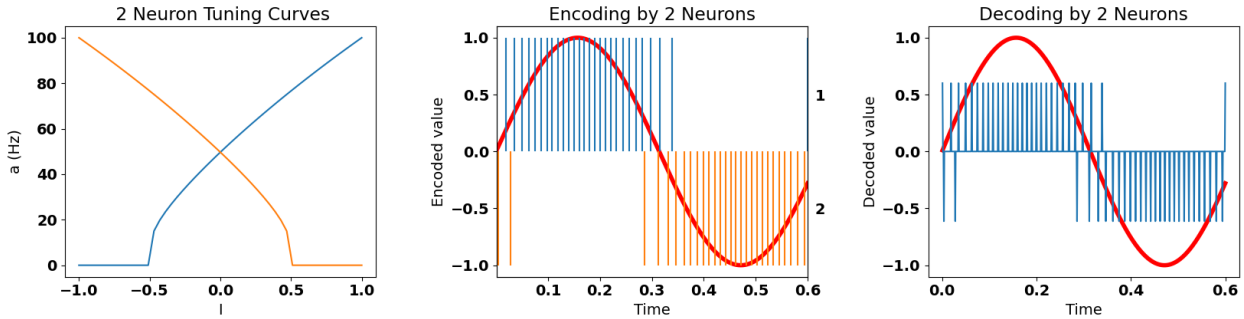
עקומת כוונון (באנגלית: Tuning Curve), היא עקומת I-F שמתארת את רמת הפעילות a של נוירון עבור כל גירוי שמתקבל כזרם קלט I, כאשר הפעילות היא למעשה תדירות f שמתייחסת לקצב ירי הספייקים על-ידי אותו נוירון. כלומר, עקומות אלה מראות כיצד תדירות ירי הספייקים של נוירון משתנה כתלות בקלט שלו. ב-Nengo, כל נוירון מתואר על-ידי עקומת כוונון משלו שמוגדרת על-ידי הפרמטרים $encoder$, שמאפשר לשלוט על אופן הקידוד של הערך שהנוירון מקבל, ו-$intercept$ שקובע את הערך בו הנוירון מתחיל להגיב (להיות פעיל). אנסמבל של נוירונים הוא למעשה אוסף של עקומות כוונון. עקומות הכוונון חיוניות להבנת האופן בו נוירונים באנסמבל מגיבים לגירויים שונים וכיצד הם מקודדים מידע.

להלן דוגמה פשוטה לעקומת כוונון של נוירון שמוגדרת על-ידי $encoder = 1$ ו-$intercept = 1$:



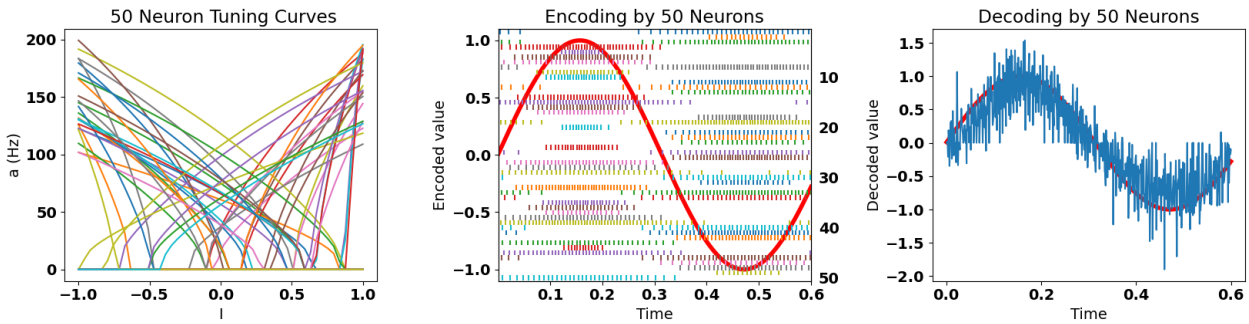
עקומה זו מגדירה נוירון שלא מגיב לגירוי קלט נמוך מ-$1\ mA$, אך מ-$1\ mA$ והלאה, מגיב בתדירות ירי ספייקים שהולכת ועולה (בקצב שהולך ויורד). תדירות ירי הספייקים של הנוירון משקפת את רמת הפעילות שלו, ולכן נאמר שהנוירון פעיל החל מגירוי קלט של $1\ mA$.

על-מנת לייצג מבנים מתמטיים מורכבים יחסית, נדרשים מספר נוירונים, ולכן ניתן באמצעות Nengo ליצור מספר עקומות כוונון. התרשימים הבאים מציגים כיצד אנסמבלים של שני נוירונים, ו-50 נוירונים, מתוארים על-ידי עקומות כוונון, מקודדים קלט סינוסי, ומשחזרים אותו.



בצד שמאל ניתן לראות שתי עקומות כוונון עם $intercept$ של 0.5 ומקודדים 1 עבור הנוירון הראשון, ו--1 עבור הנוירון השני. באמצע ניתן לראות כיצד שני הנוירונים מקודדים את הקלט הסינוסי, ובצד ימין ניתן לראות את הפיענוח של הקלט באמצעות שני הנוירונים. ניכר שהתוצאה קרובה לא למקור.

נגדיר נוירונים נוספים על-ידי יצירה של עקומות כוונון אקראיות ונקבל:



בצד שמאל ניתן לראות עקומות כוונון שונות עבור 50 נוירונים. באמצע ניתן לראות כיצד שני הנוירונים מקודדים את הקלט הסינוסי, ובצד ימין ניתן לראות את הפיענוח של הקלט באמצעות 50 הנוירונים. ניכר שהתוצאה מחקה את ההתנהגות של הקלט המקורי אך בצורה לא מדויקת. כדי להעלות את רמת הדיוק נדרשות מניפולציות נוספות כמו שימוש במסננים ואופטימיזציה של בחירת עקומות הכוונון.

להלן קישור לקוד הסימולציה:

github.com/shlomi1993/Neuromorphic-Engineering/blob/main/nengo/HW/tuning_curves.py

סעיף ו':

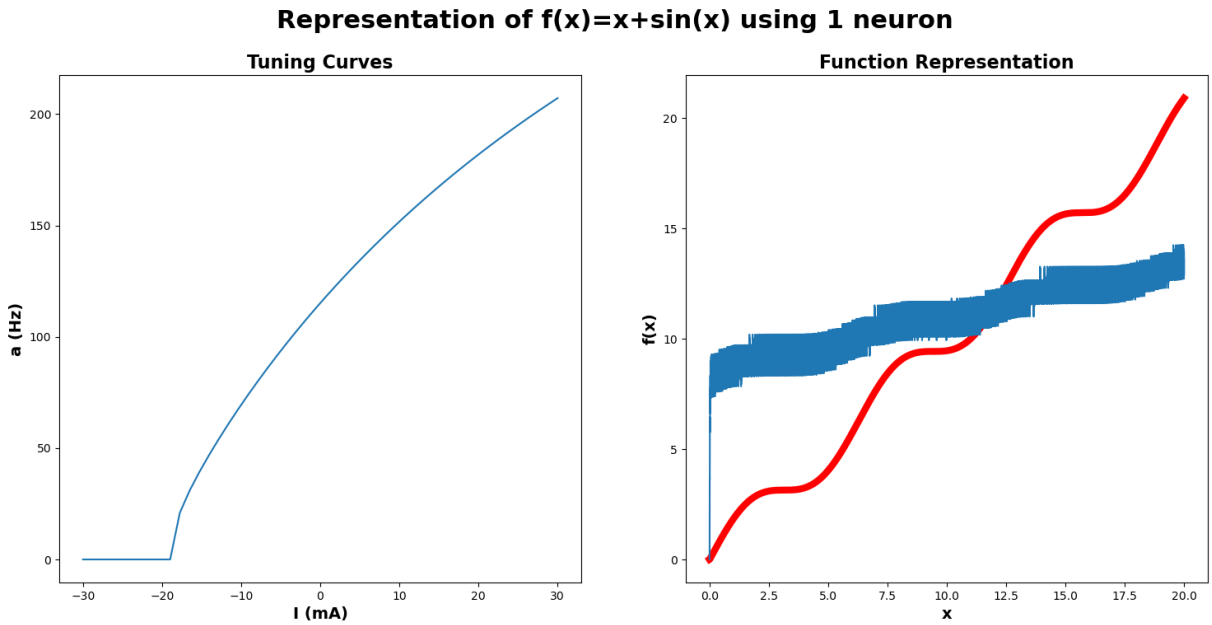
הדגימו ייצוג של הפונקציה $x + \sin(x)$ באמצעות 1, 10, 50 ו-1000 נוירונים. קבעו עקומות כוונון רנדומליות.

תשובה:

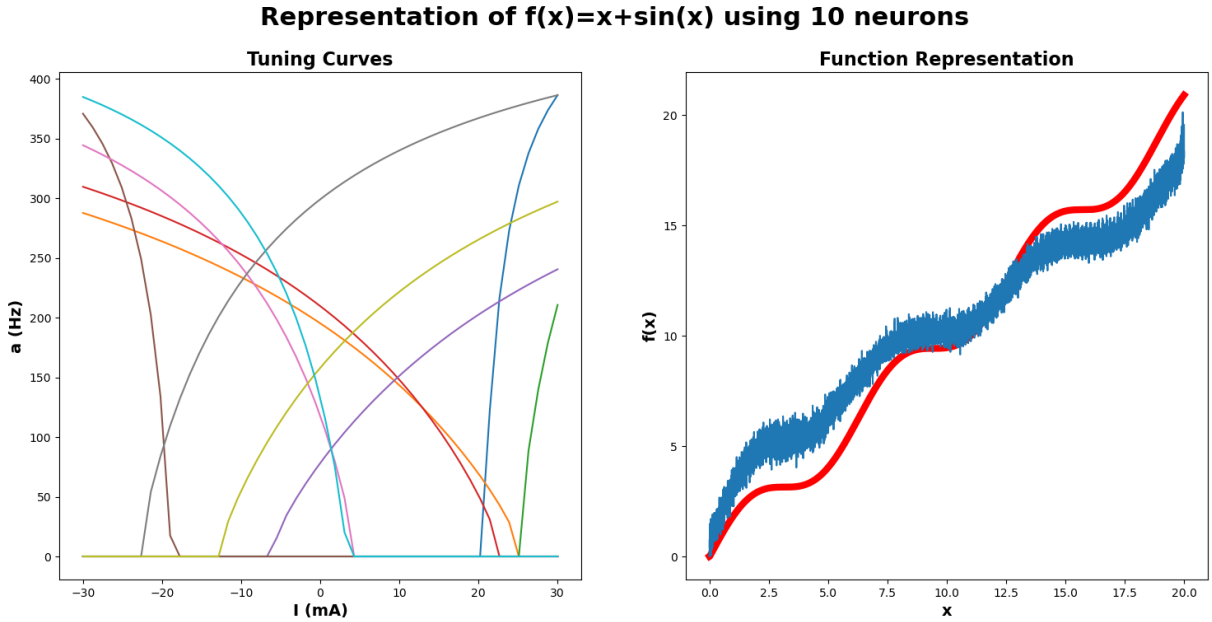
לכל מספר נוירונים $n_{neurons} \in \{1, 10, 50, 1000\}$, נגדיר מודל נוירוני על-ידי צומת קלט שפולטת ערכים לפי הפונקציה $x + \sin(x)$, אנסמבל נוירונים בגודל $n_{neurons}$ עם רדיוס 30, וצומת פלט לטובת השגת מפענחים. נחבר את הצומת הראשונה לאנסמבל, אותו נחבר לצומת השנייה, ונדגום את ערכי הצמתים עם $\tau_{synapse}$ של 0.01. נריץ את הסימולטור במשך 20 שניות (לכל $n_{neurons}$) ונקבל את התרשימים הבאים.

[המשך בעמודים הבאים...]

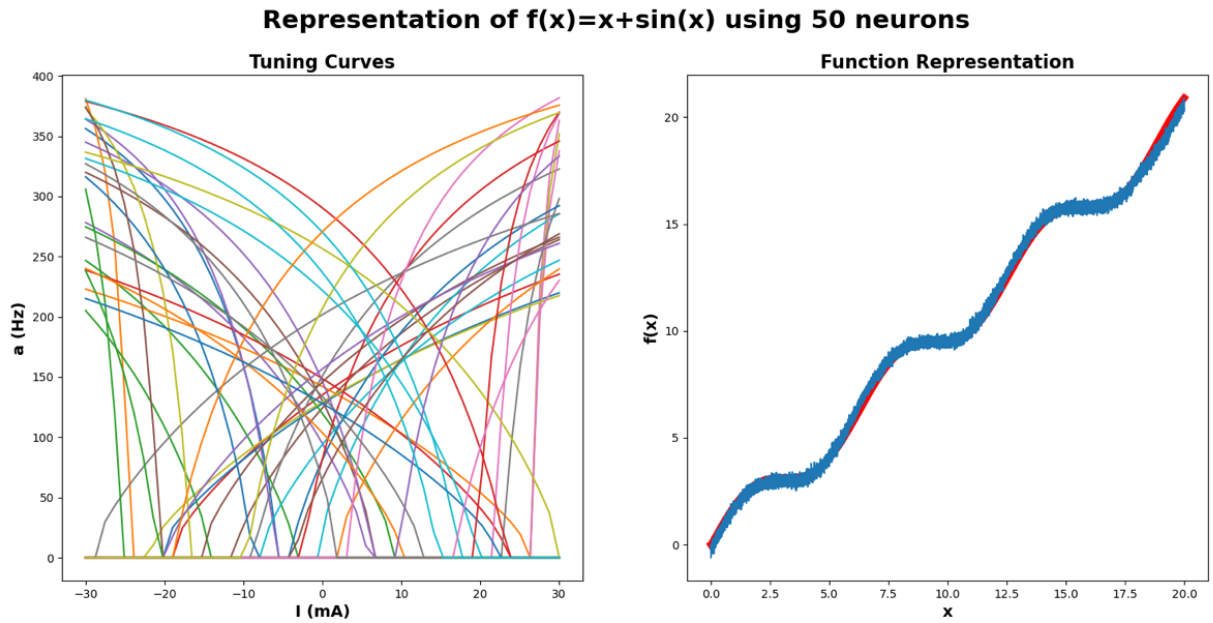
עבור $n_{neurons} = 1$ נקבל:



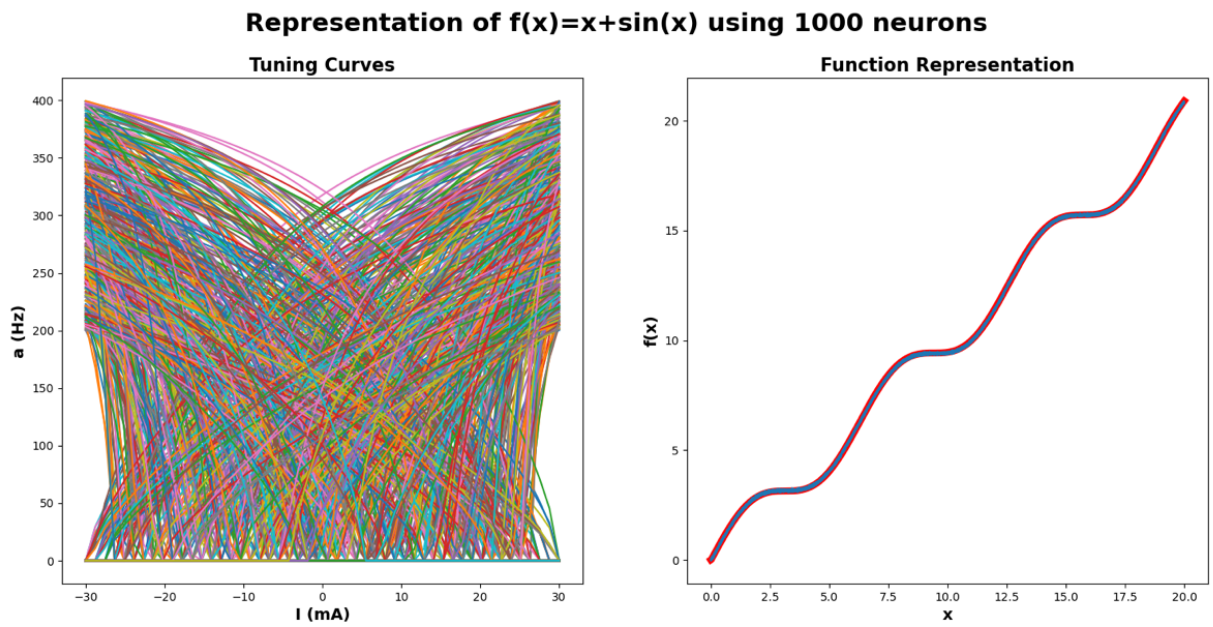
עבור $n_{neurons} = 10$ נקבל:



עבור $n_{neurons} = 50$ נקבל:



ועבור $n_{neurons} = 1000$ נקבל:



ניתן לשים לב שהוספה של נוירונים גוררת ייצוג מדויק יותר של הפונקציה המבוקשת. עם זאת, אין להתעלם מכך שככל שמוסיפים למודל נוירונים, הוא צורך יותר משאבים חישוביים.

להלן קישור לקוד הסימולציה:

github.com/shlomi1993/Neuromorphic-Engineering/blob/main/nengo/HW/tuning_curves_2.py

שאלה 2 – טרנספורמציית נתונים באמצעות NEF

סעיף א':

הסבירו בקצרה מהי טרנספורמציה. תארו שני אופנים בהם ניתן לערוך טרנספורמציה, וציינו את היתרון והחיסרון של כל אחד מהם.

תשובה:

ב-NEF, טרנספורמציה היא העיקרון השני המנחה עיצוב מודלים נוירוניים בקנה מידה גדול. מושג זה מתייחס לתהליך המרת ייצוגים נוירוניים (מקודדים בפעילויות נוירונית) מצורה אחת לאחרת, לטובת ביצוע חישוב או העברת מידע בין אוכלוסיות נוירונים. טרנספורמציות מאפשרות מימוש של פונקציות לינאריות או לא-לינאריות של אותות מקודדים באמצעות קשרים סינפטיים ממושקלים בין אנסמבלים של נוירונים. מימושים אלה מהווים למעשה את החישוב הנוירומורפי, ולכן לעתים עיקרון זה נקרא גם חישוב (באנגלית: compute).

להלן שני אופנים לביצוע טרנספורמציה:

1. טרנספורמציה לינארית – טרנספורמציה של פעילות נוירונית מקודדת אחת לאחרת באמצעות הכפלה במטריצה כלשהי באופן לינארי. מטריצה זו מגדירה את האופן בו פעילות נוירונית של אנסמבל אחד משפיעה על הפעילות של אנסמבל אחר.

מתמטית, יהיו אנסמבלים x ו-y. אזי ניתן לקודד את x באמצעות הפעילויות $a_i = G_i\left(\alpha_i e_i x + j_i^{bias}\right)$, ולפענח באמצעות $\hat{x} = \sum_i a_i d_i$. באופן דומה, ניתן לייצג את y באמצעות $b_j = G_j\left(\alpha_j e_j y + j_j^{bias}\right)$ ולפענח באמצעות $\hat{y} = \sum_j b_j d_j$. כדי להשיג $x = y$, נציב $y = \hat{x}$, ונקבל $b_j = G_j\left(\alpha_j e_j (\sum_i a_i d_i) + j_j^{bias}\right)$, משוואה השקולה למשוואה $b_j = G_j\left(\sum_i \alpha_j e_j a_i d_i + j_j^{bias}\right)$. נגדיר $w_{ij} = d_i \cdot \alpha_j e_j$, ונקבל:

$$b_j = G_j\left(\sum_i w_{ij} a_i + j_j^{bias}\right)$$

אם כן, נוכל להגדיר מטריצת משקולות W, שמחזיקה קומבינציית מכפלות של המפענחים d_i של האנסמבל הראשון, והמקודדים e_j של האנסמבל השני, באופן הבא:

$$W = d \otimes e = \begin{bmatrix} d_1 e_1 & d_1 e_2 & \dots & d_1 e_m \\ d_2 e_1 & d_2 e_2 & \dots & d_2 e_m \\ \dots & \dots & \dots & \dots \\ d_n e_1 & d_n e_2 & \dots & d_n e_m \end{bmatrix}$$

היתרון המרכזי של הטרנספורמציות הלינאריות הוא הפשטות. יתרון זה בא לידי ביטוי הן במימוש ובהבנה שלהן, והן ביעילות שלהן מבחינה חישובית. טרנספורמציות אלה מוגדרות היטב מבחינה מתמטית, דבר שמאפשר לנתח אותן בקלות יחסית.

החיסרון המרכזי של הטרנספורמציות הללו הוא יכולת הביטוי המוגבלת שלהן. הן לא מסוגלות לתפוס קשרים לא-לינאריים בין משתנים, מה שמגביל את יכולתן למדל פונקציות מורכבות, לא-לינאריות או טמפורליות, שנדרשות לעתים קרובות בחישובים נוירוניים.

2. <u>טרנספורמציה לא-לינארית</u> – טרנספורמציה של פעילות נוירונית מקודדת אחת לאחרת באמצעות מיפויים, פונקציות או אלגוריתמים המממשים פעולות לא-לינאריות. טרנספורמציות אלה נמצאות בשימוש תדיר ברשתות נוירונים מתקפיות.

מתמטית, כדי לחשב טרנספורמציה לא לינארית של x ל-y, נשתמש בפונקציה לא-לינארית f. נחליף במשוואה $\hat{x} = \sum_i a_i d_i$ את המפענחים d_i ב"מפענחי טרנספורמציה" $d_i^{f(x)}$ ונקבל $\widehat{f(x)} = \sum_i a_i d_i^{f(x)}$.

כלומר, נשתמש בפונקציה הלא-לינארית f כדי למפות פיענוח של ערך קלט מקודד לפלט.

היתרון המרכזי של הטרנספורמציות הלא-לינאריות הוא יכולת הביטוי שלהן. באמצעותן, ניתן לייצג מגוון רחב של פונקציות וקשרים בין אנסמבלים, מה שמאפשר ליצור מודלים נוירוניים יותר מורכבים, ויותר סבירים ביולוגית.

החיסרון המרכזי של הטרנספורמציות הללו הוא שהן מאוד מורכבות. הן יקרות יותר מבחינה חישובית, ומסובכות למימוש, להבנה, ולניתוח באופן מתמטי, ולרוב נדרשות עבורן טכניקות אופטימיזציה שונות.

לסיכום, טרנספורמציה היא העיקרון הבסיסי השני ב-NEF שמנחה עיצוב מודלים נוירוניים. טרנספורמציות מאפשרות להמיר ייצוגים נוירוניים מצורה אחת לאחרת, כך שבאמצעותן מושג החישוב הנוירומורפי. ישנם שני סוגים מרכזיים של טרנספורמציות – לינאריות, ולא-לינאריות – כאשר הראשונות פשוטות יותר למימוש ולחישוב, אך מוגבלות ביכולת הביטוי שלהן, והאחרונות פחות מוגבלות ביכולת הביטוי, אך בעלות חישובית גבוהה ובמימוש מסובך.

<u>סעיף ב'</u>:

הדגימו עם Nengo באמצעות plots טרנספורמציה מ-x לשלוש פונקציות לבחירתכם, באמצעות 10, 100 ו-1,000 נוירונים לאנסמבל. הסבירו את התוצאות.

<u>תשובה</u>:

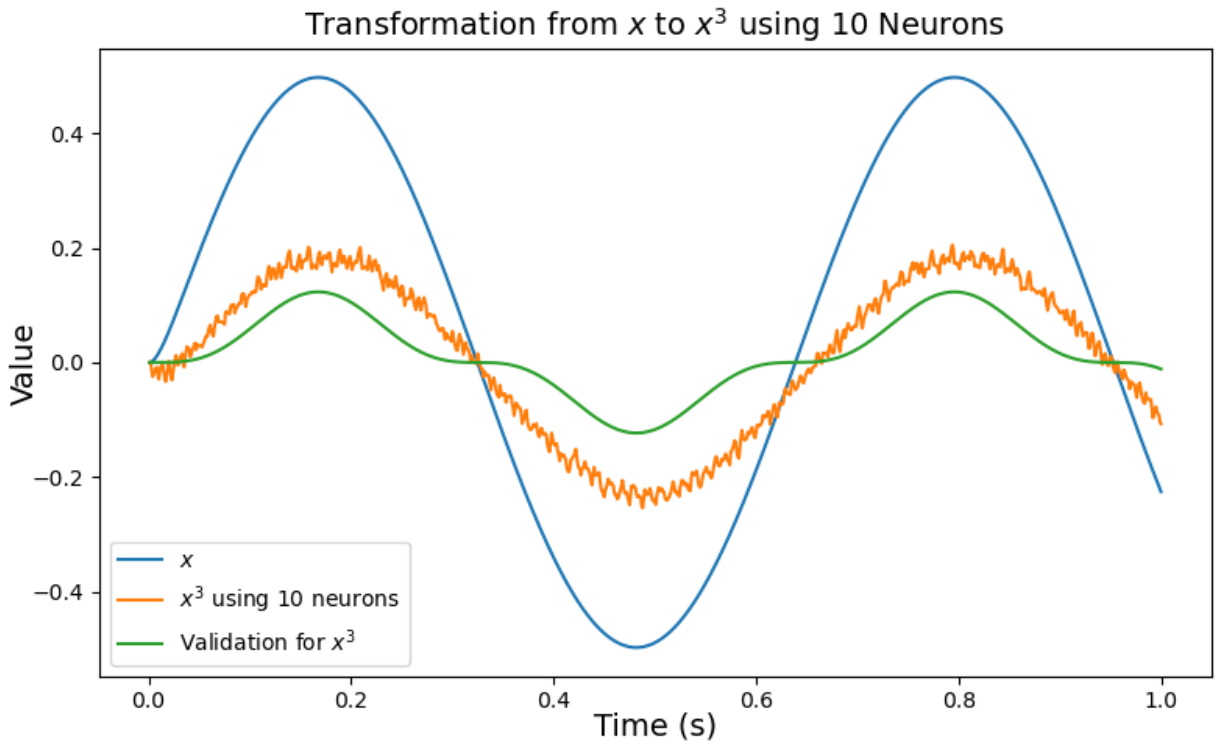
לצורך ההדגמה נבחר את שלושת הפונקציות הבאות:

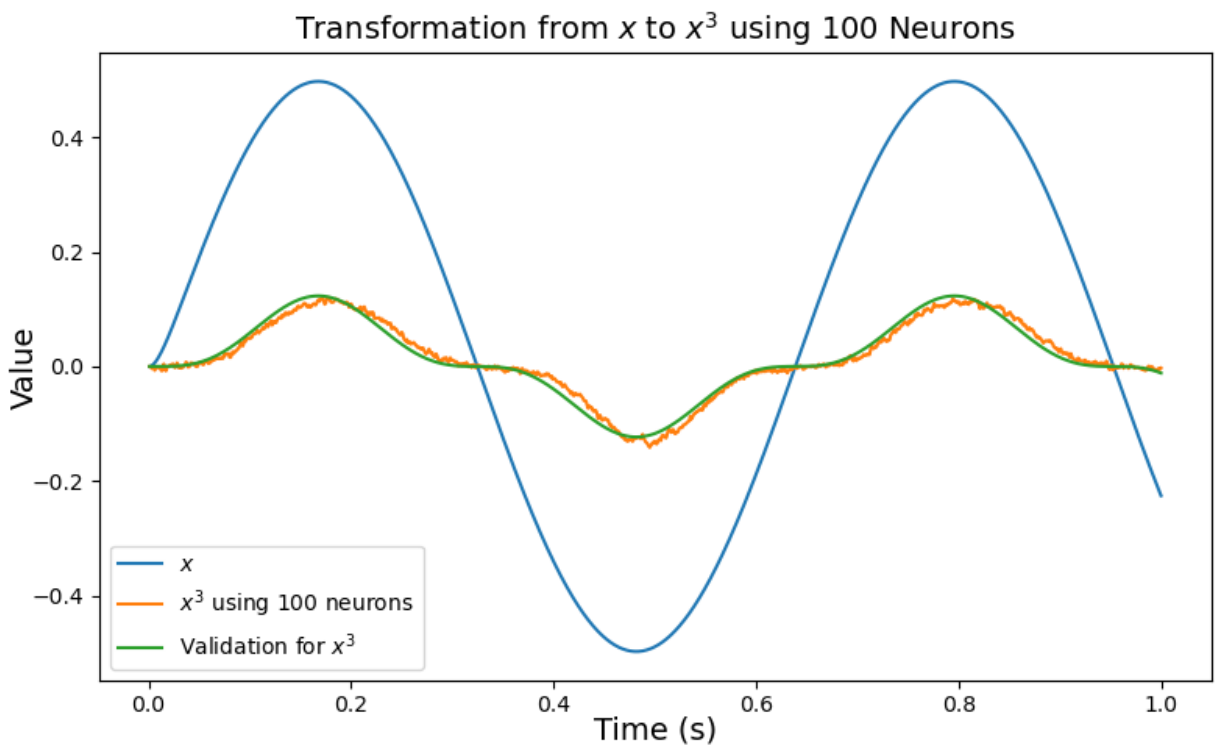
1. $f_1(x) = x^3$ – בדומה להדגמה של x^2 שבמדריך הלמידה, בתוספת ערכים שליליים.
2. $f_2(x) = \sigma(x) = \frac{1}{1+e^{-x}}$ – פונקציית הסיגמואיד.
3. $f_3(x) = \max(0, -x)$ – ואריאציה שלילית של ReLU.

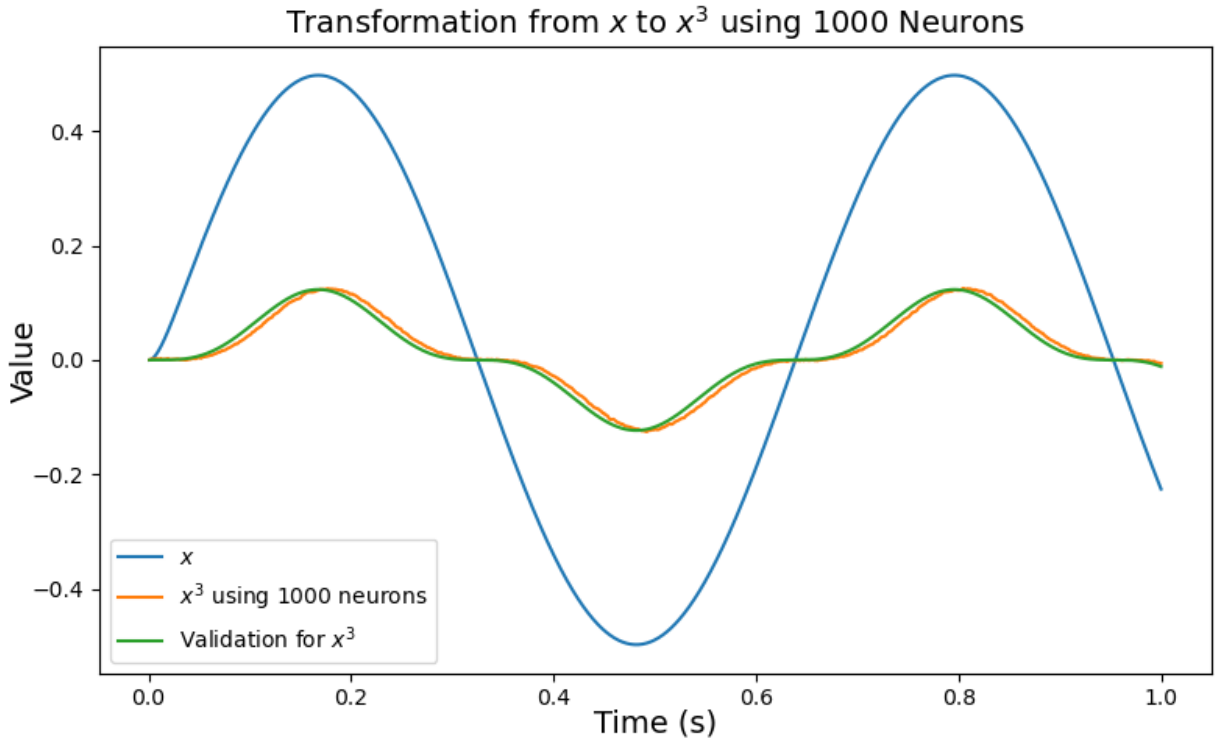
באמצעות התרשימים הבאים, נדגים טרנספורמציה מ-x ל-$f_i(x)$ עבור $i \in \{1, 2, 3\}$. בתרשימים אלה, העקומה הכחולה מתארת את ערך הקלט x, העקומה הכתומה מתארת את ערך הפלט (תוצאת הטרנספורמציה) $f_i(x)$, והעקומה הירוקה היא עקומת אימות, שנוצרה על-ידי חישוב ישיר (לא נוירומורפי) של $f_i(x)$ עבור כל x.

[המשך בעמודים הבאים...]

נדגים טרנספורמציה מ-x אל x^3:







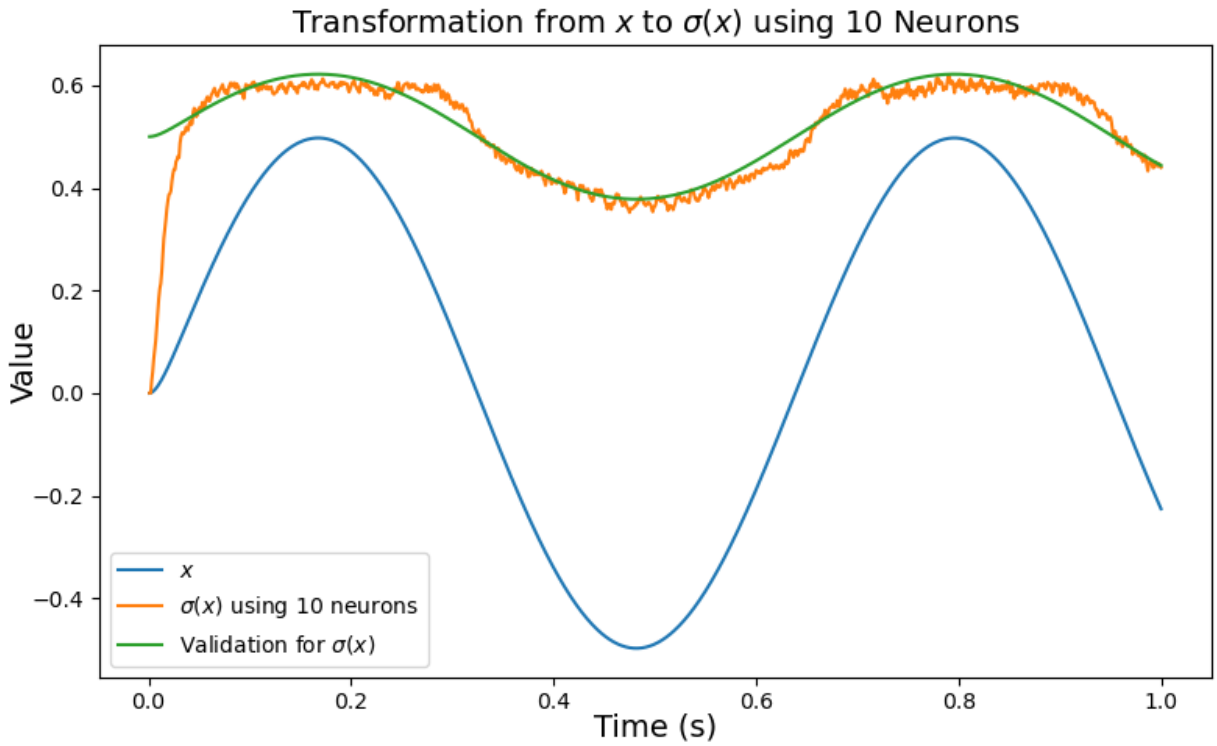
ניתן לראות שככל שמוסיפים נוירונים למודל, כך מתקבל ייצוג מדויק יותר של הפונקציה. עם זאת, ניתן לזהות היסט של עקומת הפונקציה המשוחזרת ימינה ביחס לעקומת האימות, וזאת כתוצאה משגיאת העיוות (distortion error), אשר נגרמת על-ידי המפענחים עצמם. שגיאה סטטית זו פרופורציונלית למספר הנוירונים המקודדים n, וניתנת לתיאור על-ידי:

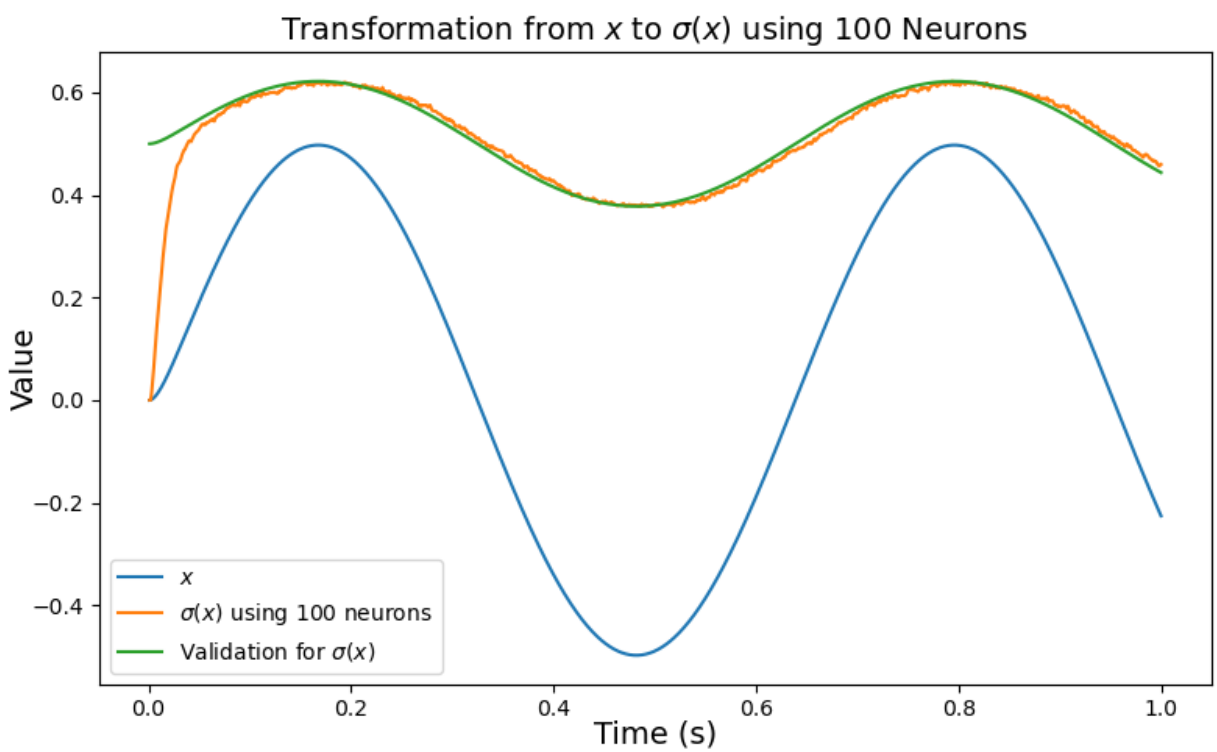
$$E_{static} = \alpha \frac{1}{n^2}$$

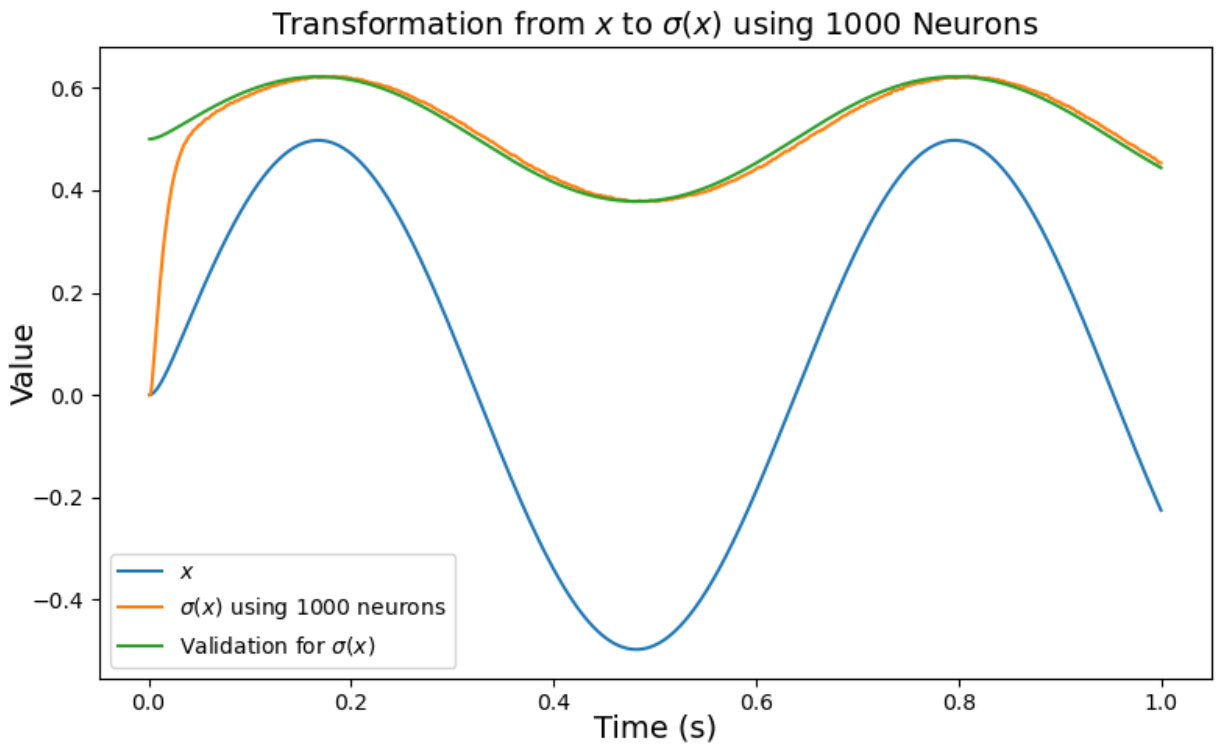
ניתן לצמצם את השגיאה על-ידי הוספה של נוירונים ואופטימיזציה בבחירה של מקודדים ועקומות כוונון, להן השפעה דרסטית על הייצוג, במיוחד בממדים גבוהים.

[המשך בעמודים הבאים...]

נדגים טרנספורמציה מ-x אל $\sigma(x) = \frac{1}{1+e^{-x}}$:



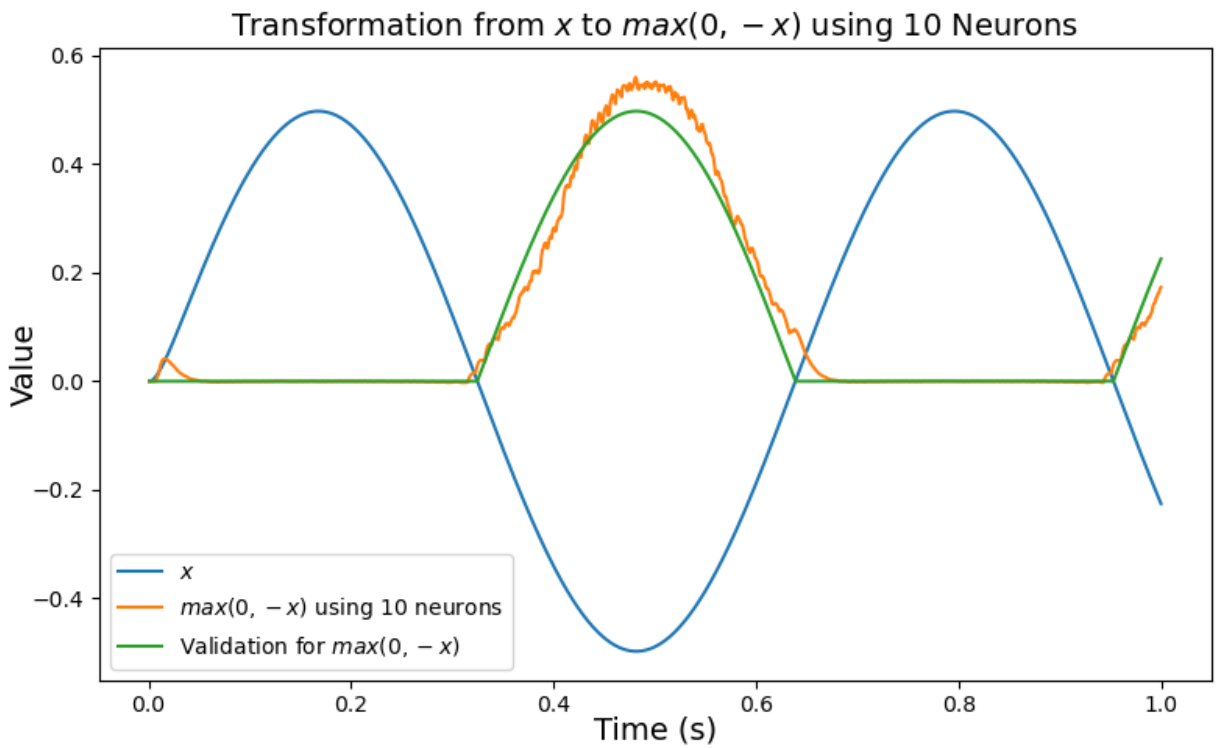


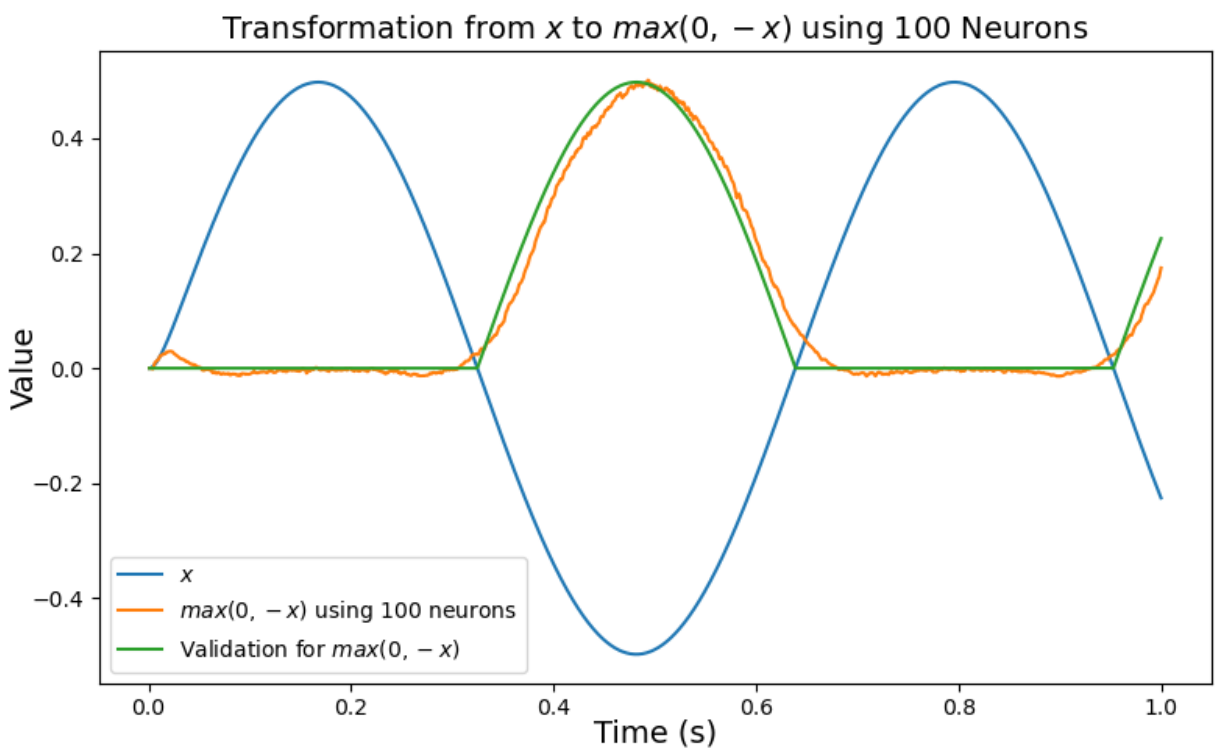


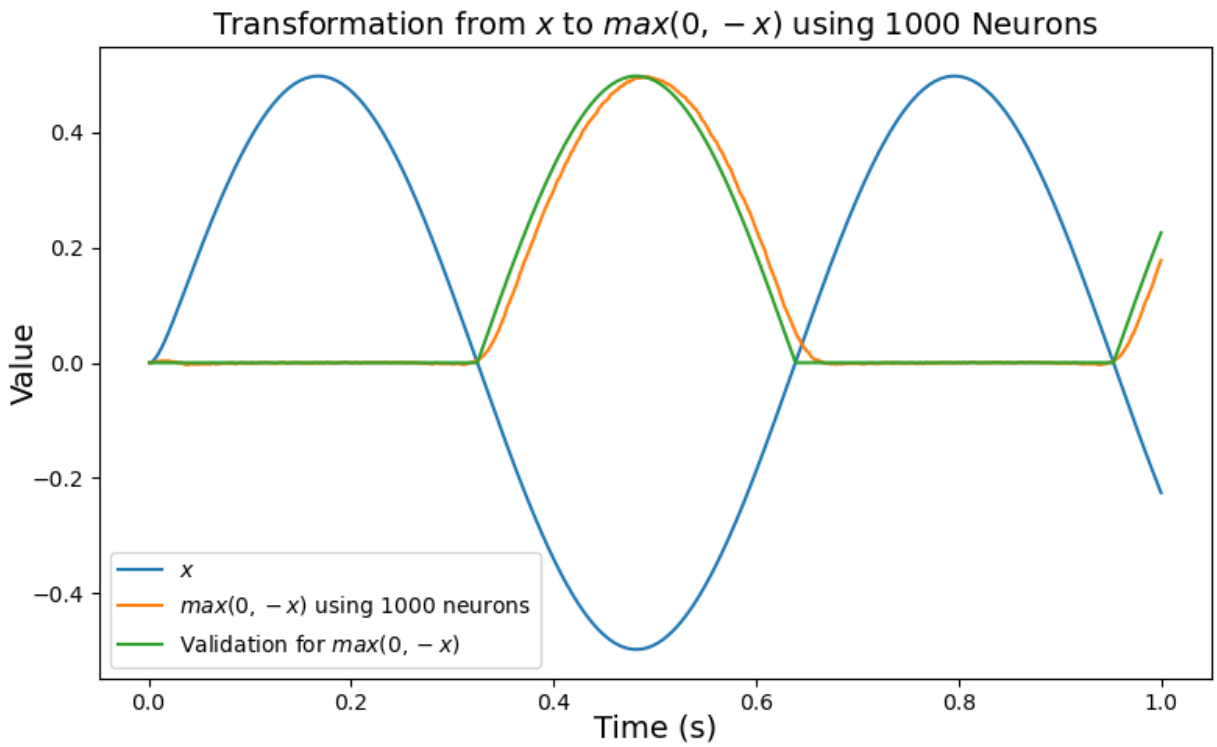
גם כאן, ניתן לזהות כיצד הגדלת מספר הנוירונים תורמת לדיוק ייצוג הפונקציה. עם זאת, בשונה מההדגמה הקודמת, ב-$t = 0$ פונקציית הסיגמואיד מחזירה 0.5 עבור $x = 0$ (ולא 0), בעוד שבנקודת זמן זו אנסמבל הנוירונים טרם למד דבר על הטרנספורמציה, ולכן ניכר אי-דיוק בתחילת הסימולציה. אי-דיוק זה הולך ומצטמצם במהירות בהמשך הסימולציה. ביישומי real-world, ניתן להתייחס לערכי הטרנספורמציה החל מזמן ייצוב (stabilization time) מוגדר מראש.

[המשך בעמודים הבאים...]

נדגים טרנספורמציה מ-x אל $\max(0, -x)$:







כמו בהדגמות הקודמות, ניתן לראות כיצד הגדלת מספר הנוירונים תורמת לדיוק ייצוג הפונקציה, וכמו כן ניתן לזהות את שגיאת העיוות.

להלן קישור לקוד הסימולציה:

github.com/shlomi1993/Neuromorphic-Engineering/blob/main/nengo/HW/transformation_hw.py

סעיף ג':

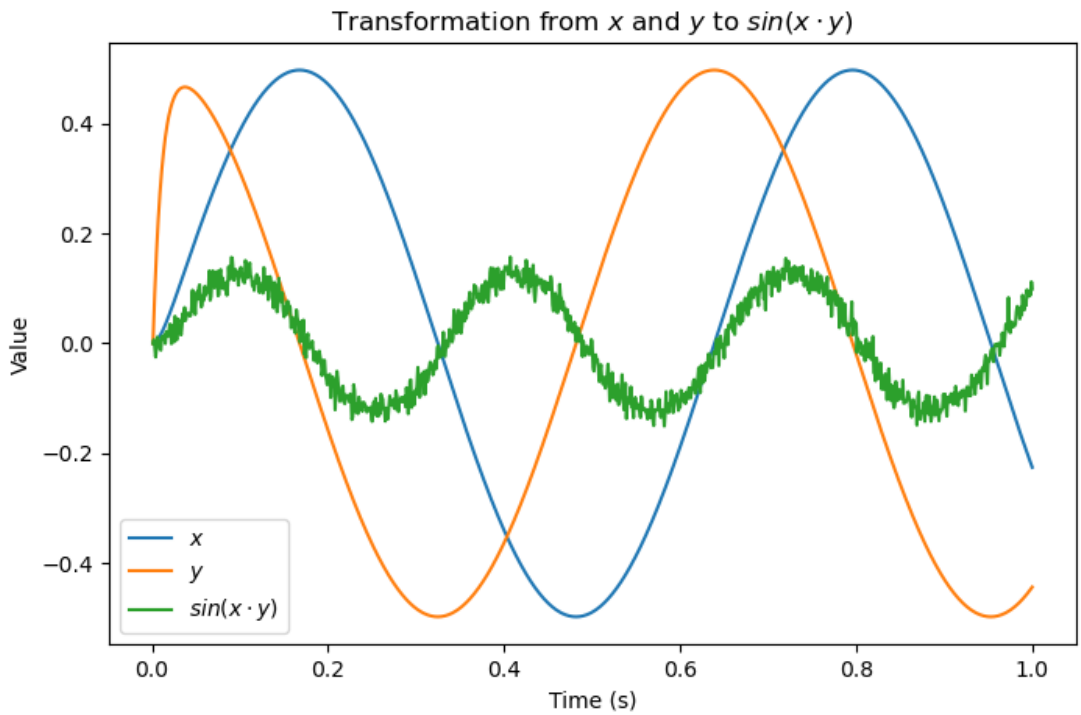
ניתן לייצג ב-Nengo ווקטורים מממד גבוה. מתי יש צורך בייצוג רב-ממדי? הדגימו את הטרנספורמציה $\sin(x \cdot y)$ באמצעות ייצוג דו-ממדי.

תשובה:

במודלים נוירוניים, ייצוג רב-ממדי נדרש במשימות בהן ייצוג חד-ממדי לא מאפשר ללכוד מורכבויות חשובות לצורך ביצוען, לרוב כי ה-data אותו רוצים לייצג או לחשב הוא מורכב מידי עבור ייצוג חד-ממדי. משימות כאלה יכולות להיות ייצוג של נתונים טמפורליים (temporal) ומרחביים (spatial) בממד גבוה, חישוב של פונקציות מורכבות (למשל במספר משתנים), ניתוח רצפים (sequential data) כמו בעיבוד שפה טבעית או פיענוח DNA, ועוד. במילים אחרות, ייצוג רב-ממדי נדרש כאשר ה-data בו עוסקים הוא רב-ממדי.

נדגים טרנספורמציה מ-x ו-y אל $\sin(x \cdot y)$ באמצעות ייצוג רב-ממדי על-ידי 1,000 נוירונים:

[בעמוד הבא...]



יש לציין שכלל לא ניתן לייצג באופן חד-ממדי את הפונקציה $\sin(x \cdot y)$ מכיוון שהיא מקבלת שני קלטים.

להלן קישור לקוד הסימולציה:
github.com/shlomi1993/Neuromorphic-Engineering/blob/main/nengo/HW/transformation_high_dim.py

שאלה 3 – דינמיקה באמצעות NEF

סעיף א':

הדגימו באמצעות Nengo מערכת דינמית הממשת מתנד (oscillator) דו-ממדי. הסבירו את המערכת במלל ובאמצעות plots רלוונטיים. כיצד ניתן לשלוט ב"מהירות" המתנד?

תשובה:

נדגים מערכת דינמית הממשת מתנד דו-ממדי, שמחזיק זוג ערכים (x_0, x_1) ומחליף ביניהם בקצב r על-ידי:

$$\frac{\delta}{\delta t}\begin{bmatrix} x_0 \\ x_1 \end{bmatrix} = \begin{bmatrix} 1 & r \\ -r & 1 \end{bmatrix}\begin{bmatrix} x_0 \\ x_1 \end{bmatrix}$$

כתוצאה מהחלפת הערכים, נוכל לקבל גרף שמתאר את הקשר בין x_0 ל-x_1 לאורך זמן כמסלולים מעגליים.

בהינתן קצב r, נגדיר את דינמיקת המתנד באמצעות המשוואות הבאות:

$$\frac{dx_0}{dt} = x_o + rx_1 \qquad \frac{dx_1}{dt} = x_1 - rx_0$$

נשים לב שניתן לשלוט ב"מהירות" המתנד על-ידי קביעת ערכים שונים עבור הקצב r, שלמעשה משמש כתדר הזוויתי של המתנד, ולכן לעיתים מסומן ב-ω.

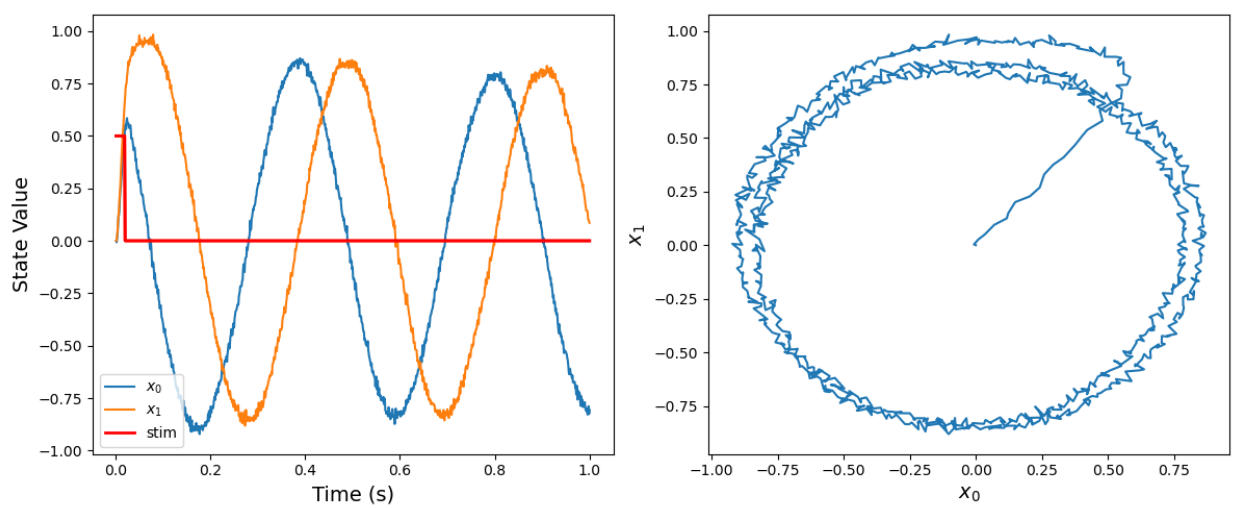
כעת, נגדיר מודל על-ידי אנסמבל שמורכב מ-200 נוירונים, אשר מייצג מצבים דו-ממדיים (x_0, x_1) של המתנד. נחבר את האנסמבל לעצמו בחיבור חוזר שמשתמש בדינמיקת המתנד שהוגדרה לעיל על-ידי יישום:

$$x_0 \leftarrow x_o + rx_1 \qquad x_1 \leftarrow x_1 - rx_0$$

כדי להימנע מ"מתנד שקט", כזה שלא זז מהנקודה (0,0), נזריק לאנסמבל קלט מצומת שיספק זוג ערכים $(0.5, 0.5)$ לפרק זמן קצר יחסית, של 0.02 שניות, ולאחר מכן יספק $(0, 0)$. כלומר, פונקציית הקלט היא:

$$I(t) = \begin{cases} (0.5, 0.5), & t < 0.02 \\ (0, 0) \ , & t \geq 0.02 \end{cases}$$

נריץ את הסימולציה עם $r = -0.15$ ונקבל את התרשימים הבאים:



בתרשים השמאלי, מוצגים הקלט באדום, וערכי x_0 ו-x_1 המתחלפים בכחול ובכתום, בהתאמה. בתרשים הימני, ניתן לראות מסלולים המעגליים שמבטאים את הקשר התנודתי שבין x_0 ל-x_1 לאורך זמן.

להלן קישור לקוד הסימולציה:

github.com/shlomi1993/Neuromorphic-Engineering/blob/main/nengo/HW/oscilator.py

[המשך בעמודים הבאים...]

<u>סעיף ב׳</u>:

סעיף זה עוסק בחישוב Ring Attractor באמצעות NEF. לממ״ן מצורף הקובץ Ring_Attractor.ipynb שמתאר ומציג Ring Attractor, הממומש באמצעות ספריות NumPy ו-SciPy. עליכם להמיר את האטרקטור המתואר בקוד המצורף לאטרקטור הממומש באמצעות נוירונים על-ידי Nengo. יש להציג את תוצאת ריצת הסימולציה בצורה ויזואלית, ולצרף למסמך את הקוד שכתבתם כטקסט. באילו קשיים נתקלתם, ומהם הפתרונות שמצאתם בבניית רשת שכזו?

<u>תשובה</u>:

הגדרת המודל

נגדיר את ה-ring attractor כמודל נוירוני באופן הבא:

1. יהיו קצבים (תדרים) $\omega_x = 1$, $\omega_y = 1.2$ ו-$\omega_z = 0.8$, כפי שהוגדר בקוד הנתון.
2. יהי מצב התחלתי רצוי $(1,0,-1)$, כמוגדר הקוד הנתון בתור ״Initial Conditions״.
3. נגדיר $\tau_{synapse} = 0.01$.
4. נגדיר צומת קלט שמוציאה פלט בערכים $(1,0,-1)$ במשך $\tau_{synapse}$ שניות, ולאחר מכן $(0,0,0)$. מתמטית, הצומת מוציאה את הפלט הבא:

$$I(t) = \begin{cases} (1,0,-1), & t \le \tau_{synapse} \\ (0,0,0) \ , & t > \tau_{synapse} \end{cases}$$

5. נגדיר אנסמבל בגודל 3,000 נוירונים מממד 3 וברדיוס 2.
   א. מספר הנוירונים נקבע בניסוי וטעיה, כמספר שמאפשר ייצוג מדויק בעלות חישובית נסבלת.
   ב. ממד הנוירונים באנסמבל נקבע כממד הקלט.
   ג. הרדיוס נקבע כך שיאפשר לייצג את כל הערכים המחושבים במהלך הסימולציה.
6. נחבר את צומת הקלט לאנסמבל, ואת האנסמבל נחבר לעצמו בחיבור חוזר שעוקב אחר פונקציה שמחשבת את ערכי x, y ו-z של המצב הבא לפי:

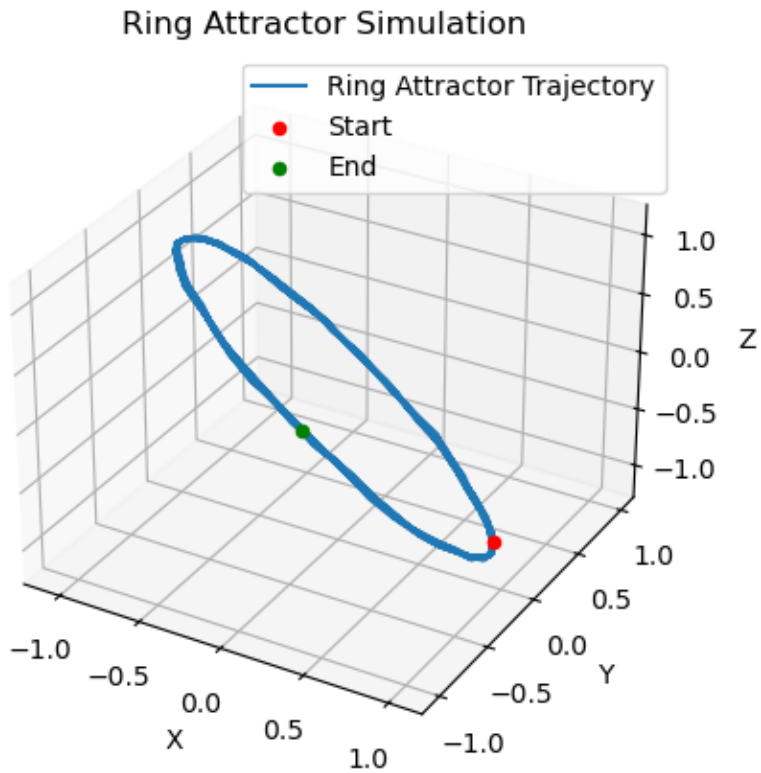
$$x \leftarrow x + \left(-\omega_y \cdot y + \omega_z \cdot z\right) \quad y \leftarrow y + \left(\omega_x \cdot x - \omega_z \cdot z\right) \quad z \leftarrow z + \left(-\omega_x \cdot x + \omega_y \cdot y\right)$$

השָׁמות אלה נקבעו לפי הפונקציה $ring_attractor$ שבקוד הנתון, אשר מממשת את המשוואות הדיפרנציאליות הבאות:

$$\frac{dx}{dt} = -\omega_y \cdot y + \omega_z \cdot z \qquad \frac{dy}{dt} = \omega_x \cdot x - \omega_z \cdot z \qquad \frac{dz}{dt} = -\omega_x \cdot x + \omega_y \cdot y$$

נשים לב כי בעוד שהקוד הנתון מחשב את השינויים בערכי x, y ו-z לאורך זמן t ומוסיף אותם לערכים הנוכחיים, על המימוש הנוירוני לחשב את הערך הבא של כל משתנה בתור הערך הנוכחי וערך השינוי כפי שהמשוואות הדיפרנציאליות לעיל מראות. זאת מכיוון שהפונקציה שמקבל רכיב ה-Connection של Nengo היא פונקציה שמחשבת את ערכי המצב הבא, ולא את ערכי השינויים.

תוצאות הסימולציה



תרשים זה מתאר את תוצאות ריצת הסימולציה, שנמשכה שתי שניות, תוך התעלמות מכל המצבים שחושבו לפני מצב ההתחלה $(1, 0, -1)$. זאת מכיוון שהאנסמבל מתחיל מייצוג המצב $(0, 0, 0)$, והדרך להביא אותו למצב ההתחלה הרצוי היא באמצעות הקלט ההתחלתי שמגיע מצומת הקלט, אליו המודל מגיב בהדרגה.

אציין שלמיטב הבנתי, לא ניתן לקבוע באמצעות Nengo את המצב שמייצג האנסמבל בתחילת הסימולציה (ללא קלט), ולכן בחרתי להתעלם ממצבי המודל עד שהוא מתייצב. אם קיימת דרך לקבוע את המצב ההתחלתי שמייצג האנסמבל, אז מוטב היה שהייתי משתמש בה.

כמו כן, יש לציין שמצב הסיום משתנה מריצה לריצה כתוצאה מעקומות הכוונון האקראיות שנוצרות בהגדרת המודל. כדי לקבל מצב סיום קבוע, ניתן להשתמש בפרמטר seed של ה-Ensemble שמאפשר ליצור עקומות כוונון אקראיות זהות בכל ריצה.

פרטי מימוש הסימולציה

```
import nengo
import numpy as np
import matplotlib.pyplot as plt

# Define tau synapse constant
tau_synapse = 0.01
```

```python
# Define input point
input_point = [1.0, 0.0, -1.0]

# Define the parameters for the ring attractor model
omega_x = 1.0
omega_y = 1.2
omega_z = 0.8

# Define the recurrent connection function
def ring_attractor(state):
    x, y, z = state
    new_x = x + (omega_z * z - omega_y * y)
    new_y = y + (omega_x * x - omega_z * z)
    new_z = z + (omega_y * y - omega_x * x)
    return new_x, new_y, new_z

# Model definition
model = nengo.Network()
with model:

    # Set the initial conditions as an initial stimulus that "turns off" after
tau_synapse seconds
    stim_func = lambda t: input_point if t <= tau_synapse else [0.0, 0.0, 0.0]
    input_node = nengo.Node(stim_func)

    # Represents a 3D state using 3000 neurons to achieve accuracy at a tolerable
computational cost
    ensemble = nengo.Ensemble(n_neurons=3000, dimensions=3, radius=2.0)

    # Connect the input node to the ensemble, and the ensemble to itself using the
ring attractor function
    nengo.Connection(input_node, ensemble, synapse=tau_synapse)
    nengo.Connection(ensemble, ensemble, function=ring_attractor, synapse=tau_synapse)

    # Attach a probe to the ensemble to measure state values
    probe_ensemble = nengo.Probe(ensemble, synapse=tau_synapse)

# Model simulation
with nengo.Simulator(model) as sim:
    sim.run(2.0)

# Extract the solution trajectories
x_values = sim.data[probe_ensemble][:, 0]
y_values = sim.data[probe_ensemble][:, 1]
z_values = sim.data[probe_ensemble][:, 2]

# Find the required starting point index
start_index = 0
```

```python
target = np.array(input_point)
closest_distance = float('inf')
for i, (x, y, z) in enumerate(zip(x_values, y_values, z_values), 1):
    current_point = np.array([x, y, z])
    distance = np.linalg.norm(current_point - target)
    if distance < closest_distance:
        closest_distance = distance
        start_index = i

# Ignore the data collected during the model stabilization time
x_solution = x_values[start_index:]
y_solution = y_values[start_index:]
z_solution = z_values[start_index:]

# Plotting the trajectories in 3D space
fig = plt.figure()
ax = fig.add_subplot(111, projection='3d')
ax.plot(x_solution, y_solution, z_solution, label='Ring Attractor Trajectory')
ax.scatter(x_solution[0], y_solution[0], z_solution[0], color='red', label='Start')
ax.scatter(x_solution[-1], y_solution[-1], z_solution[-1], color='green', label='End')
ax.set_xlabel('X')
ax.set_ylabel('Y')
ax.set_zlabel('Z')
ax.set_title('Ring Attractor Simulation')
ax.legend()
plt.show()
```

להלן קישור לקוד הסימולציה:

github.com/shlomi1993/Neuromorphic-Engineering/blob/main/nengo/HW/ring_attractor.py

אתגרים

1. **דיוק:** העקומה המתקבלת על-ידי המודל הנוירוני לא "חלקה" כמו העקומה שמתקבלת על-ידי הקוד הנתון. כדי לקבל עקומה יותר חלקה, היה צורך להעלות את רמת הדיוק של המודל הנוירוני. תחילה עשיתי זאת באמצעות הוספה של נוירונים למודל, אך מספר גדול מידי של נוירונים גרם לחישוב להיות איטי במיוחד. פתרתי זאת על-ידי מציאת מספר נוירונים גבוה מספיק לטובת הדיוק, ונמוך מכדי לגרום לחישוב להיות איטי מידי. כמו כן, מציאה של $\tau_{synapse}$ מתאים אפשרה גם היא לשפר את דיוק העקומה.

2. **מצב התחלה:** כאמור, בעוד שהקוד הנתון מתחיל את החישובים ממצב התחלה $(1, 0, -1)$, האנסמבל של המודל הנוירוני מתחיל מייצוג של המצב $(0, 0, 0)$ ומקבל את מצב ההתחלה הרצוי מצומת הקלט, מה שגורם לעקומה להתחיל מהנקודה $(0, 0, 0)$ ולהגיע ל-$(1, 0, -1)$ בהדרגה. לאחר מכן, המודל מיישם את ה-ring attractor כראוי. כפי שצוין לעיל, לא מצאתי דרך לגרום לאנסמבל לייצג את המצב $(1, 0, -1)$ עם תחילת הסימולציה, ולכן פתרתי זאת על-ידי התעלמות מהמצבים שמחושבים לפני מצב ההתחלה הרצוי.

3. **סגירת מעגל:** בתור התחלה, ניסיתי ליצור חיבור חוזר באמצעות הפונקציה $ring_attractor$ הממומשת בקוד הנתון (Ring_Attractor.ipynb) כפי שהיא, וזה גרם לעקומה הנוצרת להיות שונה מאוד מזו שמייצר הקוד הנתון, ובייחוד גרם לעקומה לא לסגור את המעגל. לאחר הבנה היטב של הקוד הנתון והתבוננות בדוגמאות שניתנו במדריך הלמידה (ובסרטונים המצורפים מ-Nengo Summer School), הבנתי שעליי לספק לרכיב ה-Connection פונקציה שמחשבת את ערכי המצב הבא בהינתן ערכי המצב הנוכחי, ולא פונקציה שרק מחשבת את השינוי. ערכתי את המימוש של $ring_attractor$ כך שתוסיף את ערכי השינוי לערכים הנוכחיים (כמתואר לעיל), וקיבלתי עקומה דומה יותר לרצויה.

4. **טווח ערכים שגוי:** הערכים שהתקבלו על-ידי הקוד הנתון עבור x, y ו-z היו כולם בסקאלה של -1 עד 1, כפי שניתן היה לראות ב-plot שקוד זה מייצר. לעומת זאת, הערכים שהתקבלו על-ידי המימוש שלי היו בטווח ערכים שונה לגמרי (למשל ציר z היה שלילי בלבד, ציר y היה רחב יותר, וכדומה). הבנתי שיש בעיה בטווח הערכים שהאנסמבל מסוגל לייצג, ולכן כדי לפתור זאת הגדלתי את הרדיוס שלו ל-2, וזה עבד, כלומר קיבלתי טווחי ערכים דומים לאלה שנותן הקוד הנתון.

5. **הישארות במסלול:** בזמן המימוש התקשיתי לגרום לאוסילטור להישאר בנתיב שלו, וניסיתי לפתור זאת על-ידי משחק עם הערכים של $\tau_{synapse}$ ושל הרדיוס. כדי לראות את ההשפעה של הפרמטרים הללו על העקומה בצורה דטרמיניסטית (שלא משתנה מריצה לריצה), השתמשתי בפרמטר seed של ה-Ensemble, שגרם לעקומות הכוונון האקראיות להיות קבועות בין ריצה לריצה, מה שאפשר לי לראות טוב יותר את ההשפעה של הפרמטרים.